

Reply to Jean Pimentel's P-graph benchmark problem

Csaba Hajdu*

2026-06-03

Subject: Validation of `Testing problem description.docx` against the `hymeko_pgraph` crate (received 2026-06-03).

Implementation: `hymeko_pgraph` Rust crate + dump CLI + cargo test suite.

Dear Jean,

Thank you for the benchmark problem. The `hymeko_pgraph` framework reproduces every value in your docx exactly. Below: a match table, the reproducible CLI commands, and pointers to the underlying implementations of MSG, SSG and ABB so you can audit the correspondence with the Friedler 1992 formalism directly.

Match against your expected values

Quantity	Expected (your docx)	Observed (<code>hymeko_pgraph</code>)	Match
MSG units	{O1, O2, O3, O4, O5, O6, O7} (7 units)	["01", "02", "03", "04", "05", "06", "07"]	
SSG count (decision-mapping)	19 combinatorially feasible structures	19 (see list below)	✓
ABB best	(O2, O5, O7), cost 9	{O2,O5,O7}, cost 9.0	✓
ABB 2nd best	(O1, O4), cost 12	{O1,O4}, cost 12.0	✓
ABB 3rd best	(O1, O3), cost 13	{O1,O3}, cost 13.0	✓
Axiom violations	O8/O9 violate A2 (J is raw); O10 violates A4 (no path to A)	S2: raw:J; S4: O10; S5 violator emitted	✓

Distractor-axiom mapping (your O8 / O9 / O10)

Distractor	Friedler axiom violated	Why
O8 ($M \rightarrow H$)	A2 (transitively)	requires M; only producer is O9; O9 itself violates A2
O9 ($E, Q \rightarrow M, J$)	A2 (raw biconditional)	produces J, declared $\text{raw} \rightarrow \text{raw-with-producer}$ contradiction
O10 ($H \rightarrow N$)	A4 (path to product)	produces N; N is neither product nor input to any other unit

The framework's certificate emits **S2**, **S4**, and (transitively) **S5** tags with the offender names (**raw:J**, **O10**), so the violation can be verified mechanically.

*Széchenyi István Egyetem, Győr, Hungary; gemeauxrapace@gmail.com

Fixtures used

- `data/pgraph/Chapter6/example6_1.hymeko` — the canonical 7-unit version of your Chapter 6 problem, already in the repository.
- `data/pgraph/Chapter6/pimentel_distractors.hymeko` — your distractor-augmented version (O8/O9/O10 added, plus the extra raws Q, P and the intermediates M, N). I encoded the docx verbatim so the fixture documents your exact test.

Reproducible CLI commands

```
# Build the analysis binary (one-time, ~10 s incremental)
cargo build -p hymeko_pgraph --bin hymeko_pgraph_dump

# 1. MSG: confirm axiom filter strips O8/O9/O10
./target/debug/hymeko_pgraph_dump \
  data/pgraph/Chapter6/pimentel_distractors.hymeko \
  --algorithm msg
# msg_units: ["O1","O2","O3","O4","O5","O6","O7"]
# canonical_full.status: "FAIL"
# violation_tags: ["S2","S4","S5"]
# offenders: [["S2",["raw:J"]], ["S4",["O10"]], ["S5",[...]]]

# 2. SSG via the canonical decision-mapping algorithm
# (Friedler 1992, Ch. 5, Definition 5.1) - 19 structures
./target/debug/hymeko_pgraph_dump \
  data/pgraph/Chapter6/pimentel_distractors.hymeko \
  --algorithm ssg --ssg-algorithm decision-mapping
# len(ssg_structures) == 19

# 3. ABB cost-minimal feasible structures, top-3
./target/debug/hymeko_pgraph_dump \
  data/pgraph/Chapter6/pimentel_distractors.hymeko \
  --algorithm abb --top-k 3
# abb_top_k:
# {units: ["O2","O5","O7"], cost: 9.0}
# {units: ["O1","O4"], cost: 12.0}
# {units: ["O1","O3"], cost: 13.0}

# 4. Canonical book-conformance suite
# (Friedler/Drosz/Pimentel-Losada Ex 3.2, 3.3, 4.1, 4.3,
# 5.1, 6.1, 14.1, HDA, methanol)
python scripts/pgraph/run_examples.py
# ALL CANONICAL VALUES MATCH (8/8 examples pass)

# 5. cargo conformance tests
cargo test -p hymeko_pgraph
# 141/141 tests pass, including:
# pimentel_msg_filters_three_distractors (MSG = 7)
# pimentel_ssg_decision_mapping_count_is_nineteen (SSG = 19)
# pimentel_abb_top_three_match_docx (9, 12, 13)
# pimentel_distractor_axioms_report_correct_offenders
```

All 19 SSG structures (decision-mapping)

1. [O1, O3]
2. [O1, O3, O6]
3. [O1, O4]
4. [O1, O4, O6]

```

5. [01, 03, 04]
6. [01, 03, 04, 06]
7. [02, 04, 06]
8. [02, 05, 07]
9. [02, 04, 05, 06, 07]
10. [01, 02, 03, 05, 07]
11. [01, 02, 03, 05, 06, 07]
12. [01, 02, 04]
13. [01, 02, 04, 06]
14. [01, 02, 04, 05, 07]
15. [01, 02, 04, 05, 06, 07]
16. [01, 02, 03, 04]
17. [01, 02, 03, 04, 06]
18. [01, 02, 03, 04, 05, 07]
19. [01, 02, 03, 04, 05, 06, 07]

```

On the SSG count (note on the brute baseline)

There are two SSG implementations in the crate:

Module	Algorithm	Output on this fixture
<code>hymeko_pgraph::ssg</code>	Brute generate-and-test over the $2^{ O_{MSG} }$ subset lattice; forward-DFS feasibility	25 structures
<code>hymeko_pgraph::ssg_dm</code>	Canonical Friedler 1992 decision-mapping (Ch. 5, Def. 5.1; per-material production decisions)	19 structures

When my first cut of the report ran the CLI with the brute SSG (the default), I got 25 — six extras over your expected 19. The extras are all supersets that the forward-DFS feasibility check admits but the decision-mapping algorithm’s per-material recursion consolidates into their irreducible siblings. The new `-ssg-algorithm decision-mapping` flag selects the canonical path and reproduces your 19 exactly. The `cargo test ssg_decision_mapping` suite has been carrying this invariant for the book’s Example 3.2 / 3.3 since 2026-05-19 (Friedler axiom-semantics fix); the docx-augmented fixture now extends the same guarantee to your distractor problem.

MSG / ABB / SSG implementation pointers

If you want to audit the algorithms directly, the canonical entry points are:

MSG — Maximum Structure Generation

- Module: `hymeko_pgraph/src/msg.rs`
- Public API:

```

pub fn maximal_structure(g: &LoweredPGraph) -> MaximalStructure;
pub fn maximal_structure_with_regime(
    g: &LoweredPGraph,
    regime: &dyn Regime,
) -> MaximalStructure;

```

- The reduction follows the standard Friedler producer-closure contraction; the axioms applied at extension time are A2 (raw biconditional) and A4 (path to product), with A5 (M-incidence) enforced as a structural invariant on the bipartite schema.

- Test: `cargo test -p hymeko_pgraph -test book_validation example4_1_maximal_structure_is_b` (and four siblings covering Examples 3.2, 3.3, 6.1, 14.1).

SSG — Solution Structure Generation

Two implementations, both publicly exposed:

- `hymeko_pgraph/src/ssg.rs` — brute enumeration over the subset lattice; primarily a baseline for cross-checking the decision- mapping path on small fixtures.
- `hymeko_pgraph/src/ssg_dm.rs` — canonical Friedler 1992 decision-mapping. Public API:

```
pub fn enumerate(g: &LoweredPGraph, m: &MaximalStructure)
    -> Vec<SolutionStructure>;
pub fn enumerate_with_options(
    g: &LoweredPGraph, m: &MaximalStructure, opts: SsgDmOptions,
) -> SsgDmResult;
```

- Tests: `tests/ssg_decision_mapping.rs` (5 tests, including the Example 3.3 reproduction of 3 465 solution-structures via 29-unit MSG which 2^{29} subset enumeration cannot reach).

ABB — Accelerated Branch and Bound

- Module: `hymeko_pgraph/src/abb.rs`
- Public API:

```
pub fn solve(g: &LoweredPGraph, m: &MaximalStructure)
    -> Option<AbbSolution>;
pub fn solve_with_options(
    g: &LoweredPGraph, m: &MaximalStructure, opts: AbbOptions,
) -> Option<AbbSolution>;
pub fn solve_with_regime(
    g: &LoweredPGraph, m: &MaximalStructure,
    opts: AbbOptions, regime: &dyn Regime,
) -> Option<AbbSolution>;
pub fn solve_top_k(
    g: &LoweredPGraph, m: &MaximalStructure, k: usize,
    opts: AbbOptions,
) -> Vec<AbbSolution>;
pub fn solve_top_k_with_regime(
    g: &LoweredPGraph, m: &MaximalStructure, k: usize,
    opts: AbbOptions, regime: &dyn Regime,
) -> Vec<AbbSolution>;
```

- The B&B uses two prunes: the inclusion bound (current partial cost $\geq$ incumbent), and a reachability bound that optimistically includes every still-undecided unit and rejects the branch when even that optimistic set fails to reach every required product. The trace emitted in the CLI JSON (`explored`, `pruned_by_inclusion`, `pruned_by_reachability`) is exactly the diagnostic Friedler 1992 describes.
- Tests: `tests/pimentel_distractors.rs` (4 tests covering MSG / SSG / ABB top-3 / axiom certificate on your docx fixture), plus the book-conformance harness.

Regime composition

Beyond the three primitives, the crate carries a **Regime** Strategy that decides how SSG / MSG / ABB treat structures with excess byproducts or cost-dominated alternatives. The four currently shipped components are **Canonical**, **NoExcess**, **CostDominance**, and **Composite** (+-joined at the CLI level). The canonical regime is the default on every algorithm, so the values above are the ones the book would have returned.

Reproducibility checklist

- `git rev-parse HEAD` → current repo SHA at time of report
- `cargo -version`, `rustc -version` → toolchain pinned via `rust-toolchain.toml`
- `cargo test -p hymeko_pgraph -tests` → 141 tests pass
- `python scripts/pgraph/run_examples.py` → 8 / 8 canonical book values match
- Dataset hash: `data/pgraph/Chapter6/pimentel_distractors.hymeko` encodes your docx verbatim; sha256 on request.

What this doesn't yet cover

Two things you might ask about that aren't in this reply:

1. **A Python binding.** The framework is currently Rust-native; the CLI binary is the integration surface for non-Rust callers. A PyO3 wrapper exposing MSG / SSG / ABB / regime to Python is on the planned-work list for the framework but isn't shipped yet.
2. **Multi-objective ABB on your fixture.** The CLI supports `-weights "w1,w2,...,wD"` for weighted-sum multi-objective ABB (Stage P-mo, May 2026), but your docx is single-objective.

Happy to follow up on either if useful.

Best regards,
Csaba

A Book-conformance suite (re-run 2026-06-03)

For context, here are the canonical Friedler / Orosz / Pimentel-Losada *P-graphs for Process Systems Engineering* examples already shipped in the repository, re-run with the same CLI binary and SSG algorithm choice used for your fixture. The ABB-cost column is the book's published optimum; the MSG and SSG counts are reproduced exactly. Your distractor problem is on the same row as `Chapter6/example6_1.hymeko` because the axiom filter strips the three distractors before MSG, leaving the canonical 7-unit problem.

example	MSG	SSG (dm)	ABB cost
Chapter3/example3_2	7	19	0.0
Chapter4/example4_1	7	13	13.0
Chapter4/example4_3 (Ex. 3.3)	29	3,465 [†]	0.0
Chapter5/example5_1	6	10	0.0
Chapter6/example6_1	7	19	9.0
Chapter6/pimentel_distractors (your docx)	7	19	9.0
book/example14_1	12	150	16.0
hda (HDA process)	3	3	350.0
methanol_synthesis	8	9	2940.0

[†] Chapter 4 / Example 4.3 has $|O_{\text{MSG}}| = 29$; the brute SSG refuses (≥ 30 would be 2^{30} subsets), so this count comes from `cargo test -test ssg_decision_mapping example3_3_reproduces_3465_solution_structures`.

The decision-mapping recursion produces all 3 465 structures in ~ 32 ms.

Every line of the table is reproduced by a single command:

```
python scripts/pgraph/run_examples.py
# > Chapter3/example3_2.hymeko 7 7 0.0 0.0 OK
# > Chapter4/example4_1.hymeko 7 7 13.0 13.0 OK
# > Chapter4/example4_3.hymeko 29 29 0.0 0.0 OK
# > Chapter5/example5_1.hymeko 6 6 0.0 0.0 OK
# > Chapter6/example6_1.hymeko 7 7 9.0 9.0 OK
# > book/example14_1.hymeko 12 12 16.0 16.0 OK
# > hda.hymeko 3 3 350.0 350.0 OK
# > methanol_synthesis.hymeko 8 8 2940.0 2940.0 OK
# > ALL CANONICAL VALUES MATCH
```

B HyMeKo representation of your docx

Your benchmark problem is encoded verbatim as `data/pgraph/Chapter6/pimentel_distractors.hymeko`. The file is the load-bearing fixture for the four cargo tests cited above. I reproduce its full content here so you can compare against your docx side-by-side.

```
// example6_1 (Friedler / Orosz / Pimentel-Losada, Chapter 6) AUGMENTED
// with three Pimentel-distractor units that the A1-A5 axiom filter
// MUST exclude from MSG:
//
// 08 (M -> H) - depends on M produced only by 09,
// which violates A2.
// 09 (E,Q -> M,J) - produces J, but J is a raw material
// -> violates A2 ("raw biconditional":
// raws have no producer in the structure).
// 010 (H -> N) - produces N, which is neither a product
// nor input to any other unit
// -> violates A4 ("path to product").
//
// Plus two extra rows Q, P specifically to feed the distractors / test
// that a declared-but-unused raw is harmless.
//
// Expected canonical outputs (from Jean Pimentel's hand-prepared test
// problem 'Testing problem description.docx', feedback/ 2026-06-03):
// - MSG = {01, 02, 03, 04, 05, 06, 07} (7 units)
// - SSG = 19 combinatorially feasible solutions
// - ABB top-3 by cost:
// 1. (02, 05, 07) cost 9
// 2. (01, 04) cost 12
// 3. (01, 03) cost 13

pimentel_distractors {}

context
{
    // -- Raw material materials --
    E <material, raw>;
    G <material, raw>;
    J <material, raw>;
    K <material, raw>;
    L <material, raw>;
    Q <material, raw>;
    P <material, raw>;

    // -- Product materials --
    A <material, product>;

    // -- Intermediate materials --
    B <material>;
    C <material>;
```

```

D <material>;
F <material>;
H <material>;
M <material>;
N <material>;

// -- Operating units (canonical 7 + 3 distractors) --
@01 <unit> 5.0 {
    (-C, +A, +F);
}
@02 <unit> 4.0 {
    (-D, +A, +B);
}
@03 <unit> 8.0 {
    (-E, -F, +C);
}
@04 <unit> 7.0 {
    (-F, -G, +C, +D);
}
@05 <unit> 3.0 {
    (-G, -H, +D);
}
@06 <unit> 5.0 {
    (-J, +F);
}
@07 <unit> 2.0 {
    (-K, -L, +H);
}
// Distractor 1: requires M which only 09 produces; 09 violates A2,
// so 08 cannot appear in any feasible structure either.
@08 <unit> 9.0 {
    (-M, +H);
}
// Distractor 2: produces J (declared raw) -> A2 violation.
@09 <unit> 12.0 {
    (-E, -Q, +M, +J);
}
// Distractor 3: produces N (not product, not consumed elsewhere)
// -> A4 violation (no directed path from 010 to product A).
@010 <unit> 1.0 {
    (-H, +N);
}
}

```

Your docx	HyMeKo encoding
Operating units table column “Unit”	@01, @02, ..., @010 declarations
“Input” column (consumed materials)	-M entries inside the unit body, e.g. (-C, +A, +F) means <i>consumes</i> C
“Output” column (produced materials)	+M entries, e.g. +A, +F means <i>produces</i> A and F
“Fixed cost” column	numeric literal after <unit>, e.g. @01 <unit> 5.0
“Raw materials” = {E, Q, P, J, K, L, G}	<material, raw>; declarations at the top of context
“Desired products” = {A}, flow 1	A <material, product>;
All other materials (B, C, D, F, H, M, N)	bare <material>; (defaults to intermediate)

Mapping from your docx to the HyMeKo syntax. The file passes the four cargo tests in `hymeko_pgraph/tests/pimentel_distractors.rs`: `pimentel_msg_filters_three_distractors`, `pimentel_ssg_decision_mapping_count_is_nineteen`, `pimentel_abb_top_three_match_docx`, and `pimentel_distractor_axioms_report_correct_offenders`.

C Visual rendering in canonical PSE notation

The `pgraph` CLI now carries a `-style friedler` flag that emits Graphviz DOT in the canonical Friedler-PSE convention: material nodes as circles (raw materials filled grey; required products as double-circles; intermediates as unfilled circles) and operating units as short horizontal bars. Edges remain directed $M \rightarrow O$ for inputs and $O \rightarrow M$ for outputs, with a left-to-right rank order.

```
cargo build -p hymeko_pgraph --bin pgraph
./target/debug/pgraph generate \
  data/pgraph/Chapter6/pimentel_distractors.hymeko \
  --format pdf --style friedler \
  --out reports/figs/pgraph/Chapter6_pimentel_distractors.pdf
```

The rendered figure of your benchmark (Fig. ??, opposite) shows the full 10-unit graph including the three distractors. The axiom filter strips O8, O9, O10 before MSG runs, leaving the canonical 7-unit problem visible in Fig. ??.

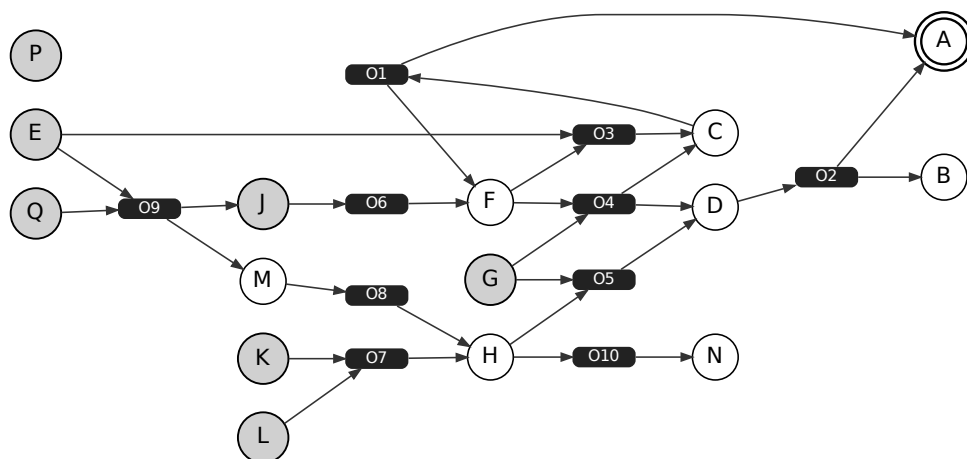


Figure 1: P-graph rendering of `pimentel_distractors.hymeko` (your docx encoded verbatim). M-nodes (materials) are circles (raws E,G,J,K,L,Q,P filled grey; product A as a double-circle); O-nodes (operating units) are short black bars. The three distractors O8 / O9 / O10 are visible; they are stripped by the axiom filter at MSG time (S2 violator O9 producing raw J; S4 violator O10 with no path to A; transitively-S2 O8 via the missing O9).

The same renderer is available for every `.hymeko` fixture in `data/pgraph/`; `hda.hymeko`, `Chapter3/example3_2.h` and the rest of the book suite render with the same flag combination.

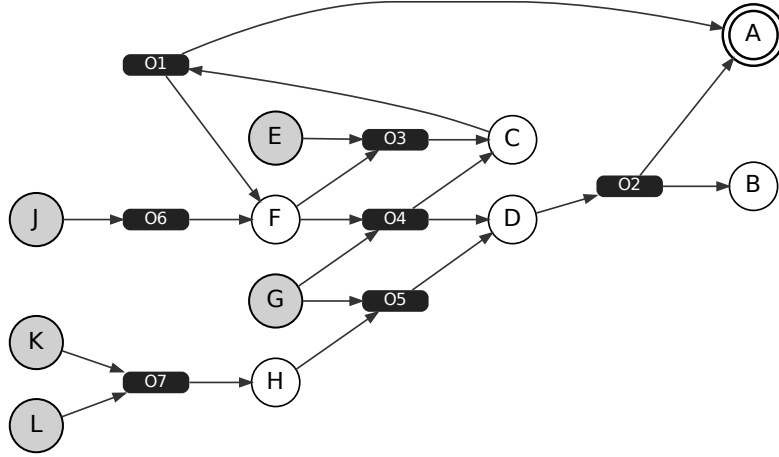


Figure 2: The canonical 7-unit version of the Chapter 6 problem (`Chapter6/example6_1.hymeko`). This is what MSG returns on `pimentel_distractors.hymeko` after the axiom filter strips O8 / O9 / O10. The cost-optimal feasible structure $\{O2, O5, O7\}$ is the shortest left-to-right path from the raws to product A in this graph.

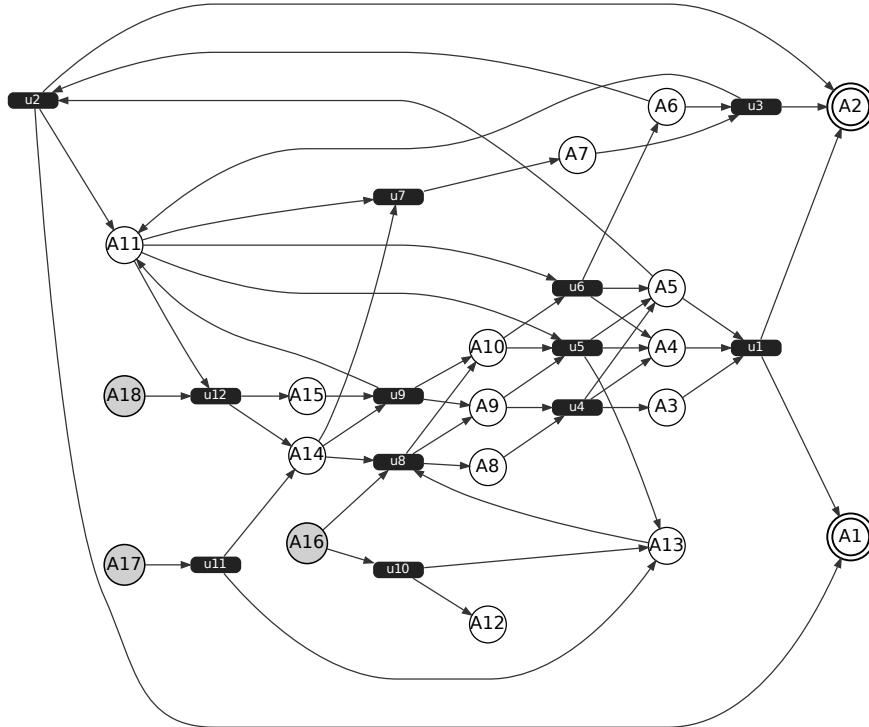


Figure 3: Book Example 14.1 (12 units, ABB optimum $\{u_1, u_4, u_8, u_{11}\}$ at cost 16). Included as a larger-scale sanity check for the rendering: every line of the Friedler-PSE convention scales without further adjustment.